

{% include header.html %}

{% include ai-share.html %}

作者：[TonTon Huang Ph.D.](#)
Blog：[2025年09月30日](#) · [vLLM](#) · [Ollama](#) · [SGLang](#) · [LLaMA.cpp](#)等四大主流熱門LLM服務框架

vLLM、Ollama、SGLang、LLaMA.cpp

深度解析四大主流熱門LLM服務框架

生產環境高吞吐與低延遲選 vLLM；本地快速上手與多模型管理選 Ollama；複雜代理/結構化工作流選 SGLang；極致輕量與可攜性選 LLaMA.cpp Server。以下從架構原理、性能優化、特性矩陣、部署與運維到選型決策提供完整分析。

框架總覽

- Ollama
 - 定位：本地多模型管理與即用即啟的服務層，重視易用性與跨平台。
 - 基礎：以 llama.cpp/ggml/gguf 生態為底，支援 CPU、Apple Silicon GPU，也可用 CUDA。
 - 強項：Modelfile 自定義、模型管理、OpenAI 兼容 API、單機體驗流暢、離線與隱私。
- vLLM
 - 定位：GPU 伺服器上的高吞吐 LLM 推理引擎。
 - 基礎：PyTorch + 自研高效 kernel，OpenAI 兼容 API，企業級佈署與擴展。
 - 強項：PagedAttention、連續批次、多 GPU 擴展、LoRA 多適配、JSON/函式工具輸出支持。
- SGLang
 - 定位：面向複雜、多步驟、可結構化的 LLM 程式化工作流引擎。
 - 基礎：LMSYS 團隊，提供前端 DSL 與高效後端（RadixAttention 等），深度整合 PyTorch。
 - 強項：跨請求 KV 重用、推測解碼、張量並行、結構化/約束輸出與多輪協調。
- LLaMA.cpp Server
 - 定位：極致輕量、單一二進位、可在任何硬體上跑的本地服務。
 - 基礎：純 C/C++ 高效實作，GGUF 量化格式；支援 CPU、CUDA、Apple Metal，甚至 WASM。
 - 強項：超輕依賴、安裝簡單、離線可用、Grammar/GBNF 約束、嵌入生成。

核心機制與性能優化

- 記憶體/快取
 - vLLM：PagedAttention 將 KV Cache 做成頁式虛擬記憶體，減少碎片與過度配置，顯著提升 VRAM 利用率與吞吐。
 - SGLang：RadixAttention 以字首樹共享 KV，適合多分支/多步驟代理流程的跨請求快取復用；並結合連續批次、零開銷排程。
 - Ollama / LLaMA.cpp：以量化縮小權重與 KV 記憶體需求，在 CPU/Metal 上仍可運作，適合長上下文但吞吐較低。
- 計算與排程
 - vLLM：連續批次動態插入新請求，保持 GPU 忙碌；支援張量/流水線並行與多 GPU。
 - SGLang：推測解碼、Chunked Prefill、張量並行；在多步驟/多路徑程式下維持高效。

- LLaMA.cpp：C++/SIMD/Metal/CUDA 核心優化；新增推測解碼、Embedding、語法約束等功能。
- 量化與精度
 - Ollama/LLaMA.cpp：主打 4/5/8-bit (GGUF)，顯著降低記憶體占用；精度受模型與量化方案影響。
 - vLLM/SGLang：支援 FP16/BF16，並逐步支援 INT4/FP8/各類 GPTQ/AWQ 等；更適合高端 GPU 場景。
- 結構化/工具調用
 - SGLang：DSL 驅動的結構化/約束生成、一致性控制與多步協調最強。
 - vLLM：支援 OpenAI 風格 JSON 模式/函式 (tool) 呼叫輸出協議，便於應用遷移。
 - LLaMA.cpp：GBNF/grammar 約束輸出成熟，對邊緣/離線結構化任務很實用。
 - Ollama：Modelfile 可預置系統提示與參數，便捷但對多步編排不著力。

四大框架特性矩陣

維度	Ollama	vLLM	SGLang	LLaMA.cpp Server
核心定位	本地便捷與模型管理	GPU 高吞吐/低延遲	複雜工作流 + 高性能	極致輕量、隨處可跑
典型硬體	CPU/Apple Silicon/NVIDIA	NVIDIA CUDA 多卡/多機	NVIDIA/部分 AMD GPU	CPU/CUDA/Apple Metal/WASM
權重/格式	GGUF、亦可導入 HF 權重	HF Transformers/safetensors	HF Transformers/safetensors	GGUF (主)
量化	4/5/8-bit (GGUF)	FP16/BF16/部分 INT4/FP8/GPTQ/AWQ	FP16/BF16/INT4/FP8/GPTQ	4/5/8-bit (GGUF)
KV Cache 優化	基於 llama.cpp 優化	PagedAttention	RadixAttention + 快取復用	高效 C++ 實作
批次/排程	基礎，單模型單隊列偏多	連續批次 + 動態排程	連續批次 + 零開銷排程	單隊列為主
多 GPU/分散式	有限	強	強 (張量並行)	有限 (以單機為主)
多模型/多租戶	易切換，併發有限	支援，多模型常駐/熱切換	支援，工作流級控制	一次多半僅載入一模型
LoRA/Adapter	基本支援	多 LoRA/PEFT 請求級	LoRA/Adapter 支援	具 LoRA 推理支援
長上下文	取決於模型與量化，速度中等	高效，適合長上下文高併發	高效，Chunked Prefill 佳	可長上下文，吞吐較低
推測解碼	限	有 (逐步完善)	有 (成熟)	有
結構化/約束輸出	基礎	JSON/函式工具模式	DSL/語法約束最強	Grammar/GBNF 成熟
多模態	取決於模型包	支援多模態模型 (視模型)	支援文字/多模態管線	取決於模型與轉換

維度	Ollama	vLLM	SGLang	LLaMA.cpp Server
OpenAI API 兼容	是	是	是	是
嵌入/向量	有	有	有	有
監控/可觀測	基礎	較完善（指標/日誌）	較完善（ workflow 視角）	基礎
部署複雜度	極低	中（需 GPU 與調優）	中（需 GPU + 程式化）	低（單一二進位）
社群成熟度	高	高	中高（增長快）	極高
代表用例	私有助手/離線/PoC	生產級 API 服務	代理/工具協作/多步任務	邊緣/離線/受限環境

性能與資源配置要點

- GPU 記憶體預估
 - 權重占用約為參數量乘每參數位元組；KV Cache 隨序列長與批次線性增長，長上下文場景請預留富餘 VRAM。
 - 量化可大幅降低權重體積，但 KV Cache 多以高精度儲存，仍是長對話瓶頸。
- 典型觀測
 - vLLM：在並發高且上下文中長時維持高 QPS 與低 P99 延遲，TTFT/TPOT 表現佳。
 - SGLang：多步/分支代理吞吐顯著優於通用引擎，純生成任務亦具競爭力。
 - Ollama/LLaMA.cpp：單機體驗流暢、tokens/s 對中小模型尚可；高併發不如 GPU 引擎。
- 取舍
 - 量化提升吞吐與可部署性，但可能犧牲部分精度與對齊品質。
 - 推測解碼在輸出熵較低時收益更大；結構化/強約束輸出會降低純生成吞吐。

部署與運維實務

- 權重與格式
 - vLLM/SGLang：偏好 HF safetensors；若來源為 GGUF 需回轉換或改用對應模型。
 - Ollama/LLaMA.cpp：偏好 GGUF；可用轉換工具從 HF 權重導出。
- 擴展與高可用
 - vLLM/SGLang：建議以容器化 + Kubernetes 水平擴展，前置一層 OpenAI 兼容閘道；支援多卡與模型分片。
 - 模型預熱與熱切換：規劃模型快取與權重上傳帶寬，降低載入抖動。
- 可觀測與治理
 - 指標：TTFT、TPOT、TPOT/P50/P99、QPS、OOM 次數、上下文長度分佈、Cost/token。
 - 控流：速率限制、輸入長度上限、並發窗口、最大批次與排程策略。
 - 安全：日誌脫敏、模型切換權限、多租戶隔離、提示註入防護在應用層實作。
- 成本優化
 - 長上下文與高併發混部：使用多池化（短上下文池/長上下文池），或使用專用草稿模型做推測解碼。
 - LoRA 多適配：vLLM/SGLang 可共用基座模型，減少多版本常駐成本。

常見情境與建議

- 個人/小型團隊、離線與隱私優先
 - 選 Ollama 或 LLaMA.cpp Server。以 4/5-bit 量化跑 7B/13B，VS Code/本地助手最省事。
- 企業級 API、生產高併發
 - 選 vLLM。規劃多卡與連續批次，設定 KV 頁面大小、Pin Memory、FlashAttention 後端，並做 P99 目標導向的壓測調參。
- 代理/工具編排、結構化輸出
 - 選 SGLang。用 DSL 管控格式、步驟與外部工具呼叫；RadixAttention 降低多分支成本。
- 邊緣/非 NVIDIA、極低依賴
 - 選 LLaMA.cpp Server。單一二進位、Metal/WASM 便利；GBNF 輕鬆產出結構化資料。

基準測試方法（實務可比性）

- 工作負載
 - 純生成（短/中/長輸入）、長上下文（ $\geq 32k$ ）、代理式多步/分支、JSON/GBNF 約束輸出。
- 指標
 - TTFT、TPOT、整體 tokens/s、QPS@P99、VRAM 佔用、OOM 次數、成本/千 tokens。
- 程式化測試
 - 以 OpenAI 兼容 API 發壓，控制溫度、top-p、一致的停止詞與格式要求；同一模型/同一權重/同一精度比對。

與其他選項的關係

- Text Generation Inference（TGI）：生產級 API 伺服器，管理功能齊全，吞吐不及 vLLM 的場景逐步被替代，但在企業治理與 Triton 集成上仍具價值。
- TensorRT-LLM/LMDeploy：更偏底層或針對 NVIDIA 最優化的路徑，極致延遲/吞吐可期，但開發/維運門檻較高。
- MLC LLM：強調跨裝置/瀏覽器部署與可攜性，與 LLaMA.cpp 取向相近。

選型決策建議（實操版）

- 你需要 GPU 上的生產級高吞吐與低延遲 → vLLM（首選），若工作流複雜或需嚴格結構化 → SGLang。
- 你需要最快落地的本地體驗與模型管理 → Ollama；若硬體極受限或需極致可攜 → LLaMA.cpp Server。
- 長上下文/多租戶/多 LoRA → vLLM 或 SGLang；JSON/GBNF 嚴格格式 → SGLang 或 LLaMA.cpp。
- 非 NVIDIA 或離線邊緣 → LLaMA.cpp 或 Ollama；Apple Silicon → Ollama/LLaMA.cpp（Metal）。

你計劃服務的硬體環境（GPU 型號/數量或僅 CPU）、目標上下文長度、P99 延遲與 QPS 指標、以及是否需要代理式多步工作流或嚴格 JSON/語法輸出？告訴我後我可給出更精準的架構與參數建議。